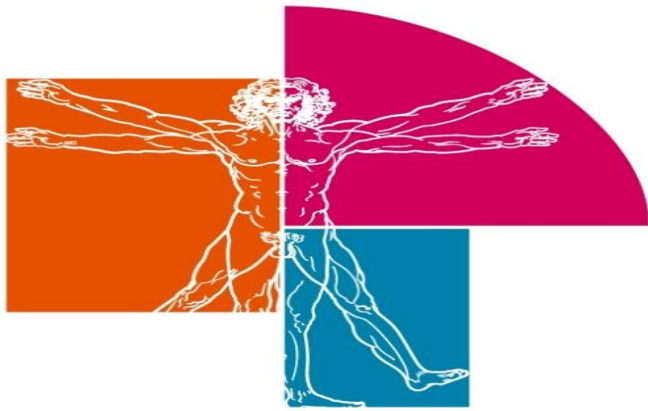




IES LeonardoDaVinci
ALBACETE



TEMA 2

INTRODUCCIÓN A LA

PROGRAMACIÓN WEB

CON JAVA



OBJETIVOS

- Conocer la plataforma J2EE
- Entender las ventajas de las aplicaciones J2EE
- Entender como se integran Java y HTML
- Conocer la sintaxis de Servlets y Jsp.
- Crear aplicaciones web Java que utilicen Servlets y JSPs.



INDICE

1. Plataforma J2EE

2. Servlets

2.1. Peticiones y respuestas

2.1.1. Objetos HttpServletRequest

2.1.2. Objetos HttpServletResponse

2.1.3. Manejar peticiones Get y Post

2.1.4. Métodos de un Servlet



INDICE

3. JSP (Java Server Pages)

3.1. Mecanismos de ejecución de JSP

3.2. Variables implícitas en JSP

3.3. Elementos de una JSP

3.4. Gestión de excepciones en JSP

3.5. Acciones en JSP

3.6. Comunicación entre páginas JSP

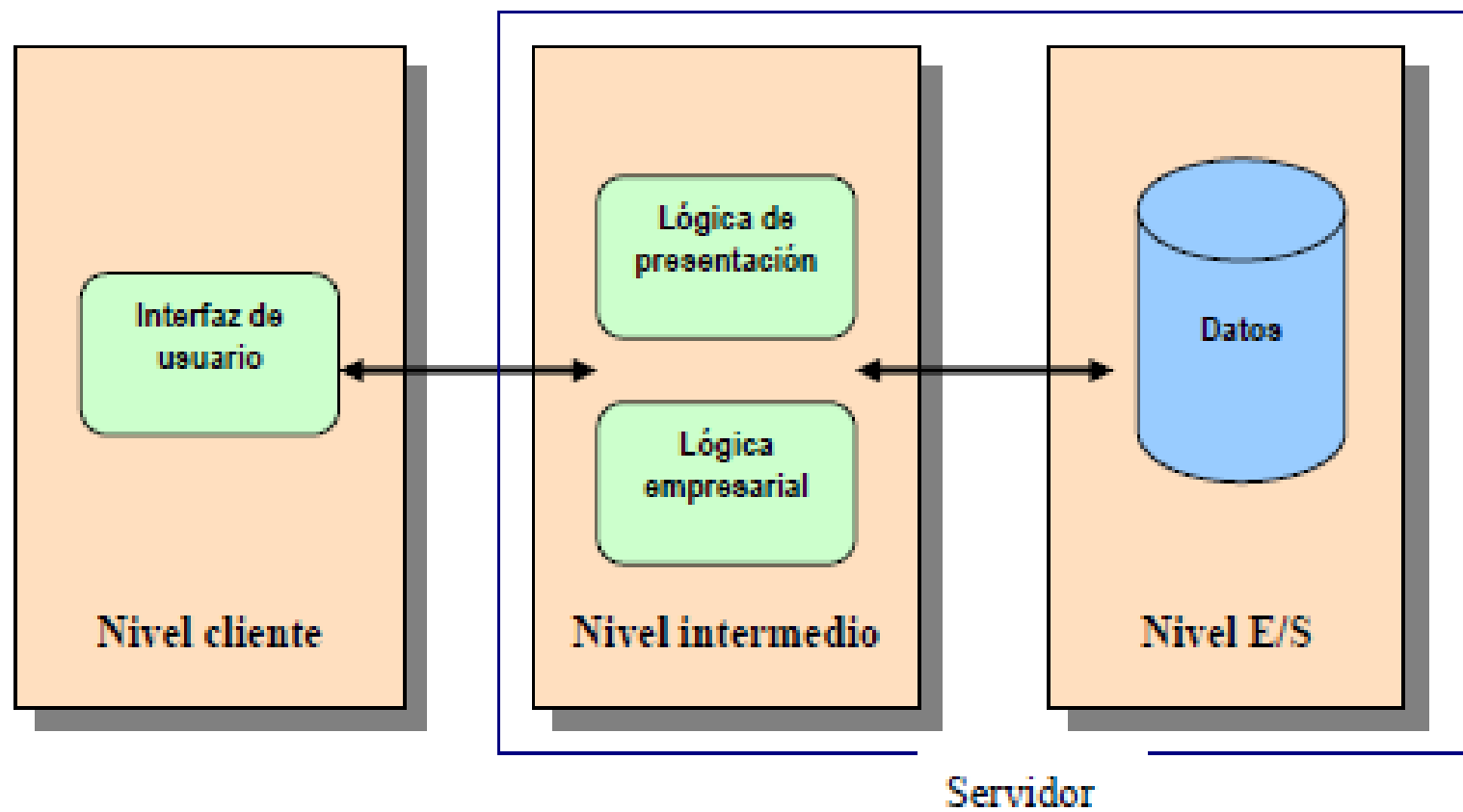


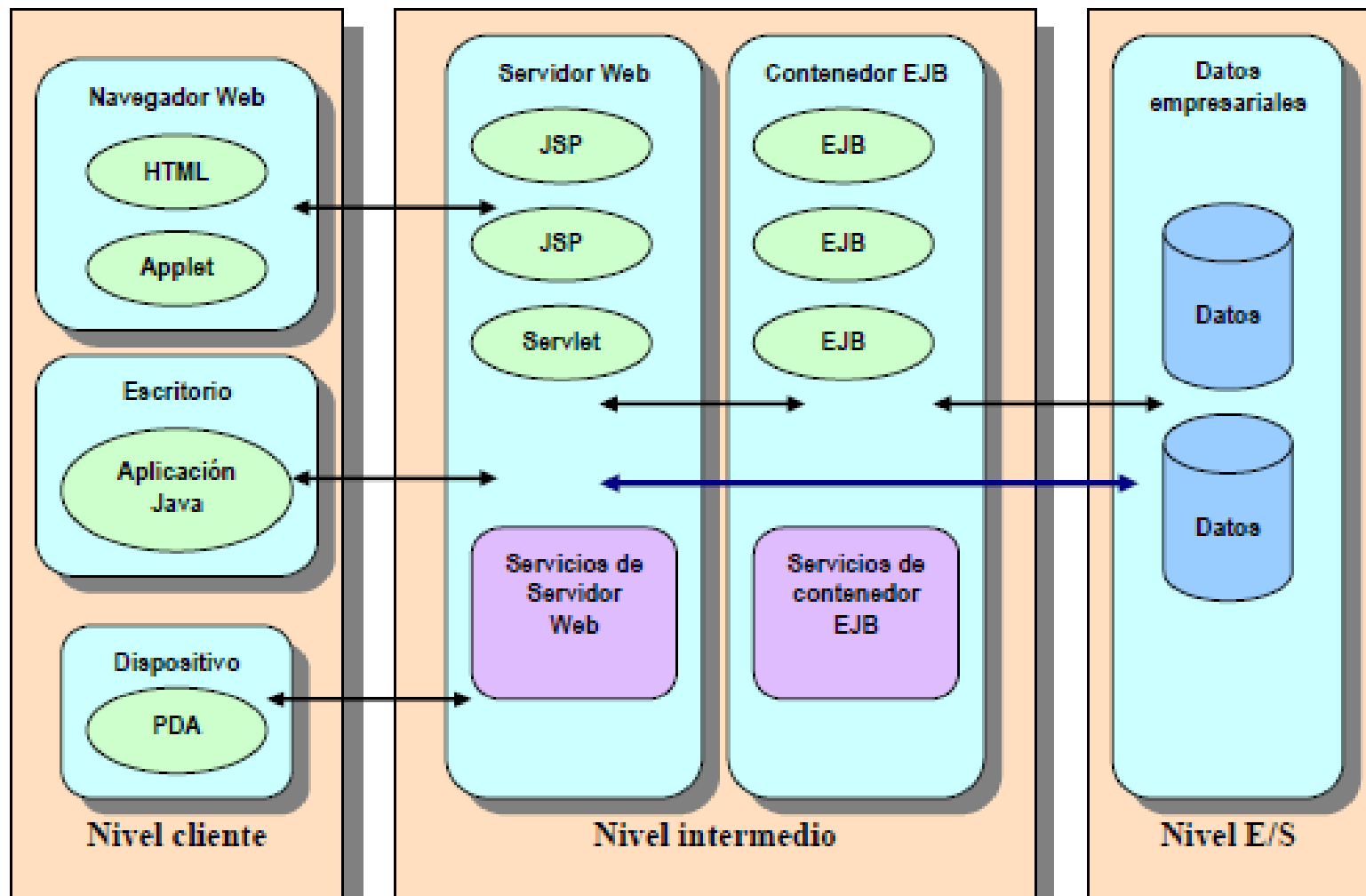
1. Plataforma J2EE

- J2EE (Java 2 Enterprise Edition) es una plataforma de **desarrollo en Java** construida **para aplicaciones empresariales (de gran envergadura) distribuidas**, desarrollada por Sun Microsystems, con aportaciones de distintos desarrolladores, incluido Borland.
- J2EE consiste en un conjunto de especificaciones, normas o directrices bajo las cuales se desarrollan aplicaciones que serán capaces de ser desplegadas y utilizadas en cualquier sistema operativo.



- J2EE incluye diferentes tecnologías como son **Java Servlet** y **JavaServer Page (JSP)** entre otras muchas.
- El desarrollo de **aplicaciones** bajo J2EE, al igual que otras plataformas como .NET, **se basa en la separación de capas**. Esta separación de capas permite una delimitación de responsabilidades a la vez que satisface los requisitos no funcionales de este tipo de aplicaciones (escalabilidad, extensibilidad, flexibilidad, etc.).







- J2EE también recomienda diseñar las aplicaciones web utilizando el patrón Modelo-Vista-Controlador, en donde:
 - Los **servlet** se usan como **Controladores**.
 - Las **páginas JSP** se usan como **Vistas**.
 - Las **clases Java**, ficheros de imagen o texto, además de las **tablas JDBC** se usan como **Modelo**.
- El servlet Controlador recibe los formularios del cliente y los procesa, enviando la petición del mismo al Modelo. De igual manera extrae datos del modelo que envía a la Vista.



- La página Vista JSP envía los datos que tiene al cliente a través de un formato html.
- Podemos añadir una nueva **capa DAO** para realizar las **acciones entre los controladores y las tablas de la base de datos.**
- La primera llamada del cliente supondrá el envío de una página .html o .jsp de inicio.



2. Servlets

- Los Servlets son **aplicaciones** que implementan los servidores **orientados a petición-respuesta**, como los servidores web compatibles con Java.
- Por ejemplo, un servlet podría recoger los datos de un formulario que se ha mandado desde un cliente remoto en HTML y aplicarle la lógica de negocios para actualizar la base de datos de la aplicación.



- Para ejecutar aplicaciones web con Java no es suficiente utilizar Apache, necesitamos disponer de un servidor de aplicaciones, como puede ser **Tomcat**. Otra alternativa es usar **Glassfish**, muy utilizado en la actualidad.
- Los servlets **no tienen interface gráfico** de usuario, pero pueden usar los componentes de las páginas HTML.
- Proporcionan una forma de **generar páginas dinámicas que son fáciles de escribir y rápidos en ejecutarse**.
- Un servlet puede manejar múltiples peticiones concurrentes, y puede sincronizarlas.



○ Ejemplo de un servlet que devuelve la hora del servidor

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.PrintWriter;
import java.io.IOException;

public class ServletHola extends HttpServlet {

    private static final String CONTENT_TYPE = "text/html; charset=windows-1252";
    public void init(ServletConfig config) throws ServletException {
        super.init(config);}

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType(CONTENT_TYPE);
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head><title>Mi primer Servlet</title></head>");
        out.println("<body>");
        out.println("<p>Hola soy un servlet.</p>");
        out.println("<br><br> La fecha del servidor es: "+new java.util.Date());
        out.println("</body></html>");
        out.close();
    }
}
```




- Comentarios al programa:

- `import javax.servlet.*`

Importa el paquete servlet, que proporciona clases e interfaces para escribir servlets.

- `import javax.servlet.http.*`

Indica que el servlet va a manejar peticiones http.

- `import java.io.PrintWriter`

Para comunicarnos con el cliente, utilizaremos el método `out.println( )`, que forma parte de la clase `PrintWriter`.



- `public class ServletHola extends HttpServlet`

La funcionalidad principal del API Servlet la proporciona la interface Servlet. **Todos los servlets implementan este interface**, bien directamente o, **más comúnmente**, extendiendo una clase que ya lo incorpora, lo implementa, como **HttpServlet**.

- `public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException`

Este es uno de los **métodos de obligada escritura** para codificar servlets, aunque actualmente lo hace el método `processRequest`.



- Este método utiliza un objeto que representa las peticiones del cliente (**HttpServletRequest**) y otro que lo hace con las respuestas del servlet (**HttpServletResponse**).
- La información del cliente que recibimos a través de la página html, está encapsulada en el objeto **HttpServletRequest**, de aquí obtenemos los parámetros pasados en un formulario para procesarlos posteriormente.
- Cuando contestamos al cliente, lo hacemos encapsulando la información en el objeto **HttpServletResponse**.
- **Es obligatorio avisar de un posible error de entrada/salida o alguno interno de servlet con la clausula throws.**



- `out.println("<br><br> La fecha del servidor es: "+new java.util.Date());`

El código que hay dentro del método tiene dos esquemas diferentes, por una parte se trata de un **formato típico de página html** utilizando un **objeto de tipo PrintWriter**, que se trasladará hasta el cliente sin ningún tipo de variación.

Por otra parte **existe código puro en java**; éste antes de enviarse al cliente **se ejecutará en el servidor** y se enviará su resultado, en este caso la fecha del sistema.

Aquí se muestra el carácter dinámico de los servlets frente al estático de las páginas html. Un servlet puede variar la respuesta al cliente dependiendo del momento en que se solicita.



2.1. Peticiones y respuestas

- Los métodos de la clase `HttpServlet` que manejan peticiones de cliente toman dos argumentos.
- Un objeto **`HttpServletRequest`**, que encapsula los datos desde el cliente.
- Un objeto **`HttpServletResponse`**, que encapsula la respuesta hacia el cliente.



2.1.1 Objetos `HttpServletRequest`

- Un objeto `HttpServletRequest` proporciona **acceso** a los **datos** de **cabecera HTTP**, cualquier **cookie** encontrada en la petición, y el **método HTTP** con el que se ha realizado la misma. El objeto `HttpServletRequest` **también permite obtener los argumentos que el cliente envía como parte de la petición.**
- El método `getParameter` devuelve el valor de un parámetro nombrado. Si nuestro parámetro pudiera tener más de un valor, deberíamos utilizar `getParameterValues` en su lugar. El método `getParameterValues` devuelve un array de valores del parámetro nombrado.



- El método **getParameterNames** proporciona los **nombres** de los **parámetros**.
- Ejemplo: Obtener todos los valores de los parámetros; cada parámetro tiene un único valor.

```
Enumeration parametros=request.getParameterNames();  
while(parametros.hasMoreElements()){  
    String param=(String)parametros.nextElement();  
    String valor=request.getParameter(param);  
    out.println(param+" = "+valor);  
}
```




2.1.2. Objetos HttpServletResponse

- Un objeto **HttpServletResponse** proporciona dos formas de **devolver datos al usuario**.
- El método **getWriter()** devuelve un **Writer**. Se utiliza para **devolver datos en formato texto** al usuario.
- El método **getOutputStream()** devuelve un **ServletOutputStream**, para **devolver datos binarios**.



○ Ejemplo:

```
package paquetecontraseña;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class ServletContraseña extends HttpServlet {

    protected void processRequest (HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
```



```
try {
    /* Business logic */
    String vUsuario = "";
    String vPassword = "";
    try {
        vUsuario = request.getParameter("Tusuario");
        vPassword = request.getParameter("Tpassword");
    } catch (Exception e) {
        e.printStackTrace();
    }
    /* Answer to the client */
    out.println("<html>");
    out.println("<head><title>Contraseña</title></head>");
    out.println("<body>");
    if (vUsuario.equals("java") && vPassword.equals("servlet"))
        out.println("<p>O.K.</p>");
    else {
        out.println("<p>ERROR</p>" +
            "<a href='index.jsp'>Volver </a>");
    }
    out.println("</body>");
    out.println("</html>");
} finally {
    out.close();
}
```



```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

@Override
public String getServletInfo() {
    return "Short description";
} // </editor-fold>
}
```



2.1.4. Métodos de un Servlet

- En Java, los servlets tienen un ciclo de vida controlado por el **contenedor web** (por ejemplo, Tomcat, Jetty, GlassFish). Dentro de ese ciclo de vida, los métodos más importantes son:
 - **init()** Se ejecuta una sola vez cuando el servlet se carga por primera vez en memoria. Sirve para inicializar recursos que el servlet va a necesitar (conexiones a base de datos, objetos compartidos, lectura de configuraciones, etc.).



```
@Override
```

```
public void init() throws ServletException {
```

```
    super.init();
```

```
    // Aquí inicializamos recursos
```

```
    System.out.println("El servlet se ha inicializado");
```

```
}
```




- **destroy()** Se ejecuta una sola vez cuando el servlet es descargado de memoria (por ejemplo, cuando se apaga el servidor o se despliega una nueva versión de la aplicación). Sirve para liberar recursos: cerrar conexiones de base de datos, hilos, sockets, etc.

```
@Override  
public void destroy() {  
    // Aquí liberamos recursos  
    System.out.println("El servlet está siendo destruido");  
    super.destroy();  
}
```



- El **ciclo de vida resumido** sería: El contenedor carga el servlet en memoria y llama a `init()`. Por cada petición HTTP, se llama a `service()`, que a su vez invoca `doGet()`, `doPost()`, etc. Cuando el contenedor decide eliminar el servlet (apagado, redeploy, timeout), llama a `destroy()`.
- Otro método interesante es **`getServletInfo`**. Este método devuelve una cadena con información descriptiva del servlet, como el autor, la versión o una breve descripción. Por defecto, devuelve `null`, pero se puede sobrescribir para documentar tu servlet.



```
@Override  
public String getServletInfo() {  
    return "InfoServlet v1.0, creado por Juan Pérez - " +  
        "Este servlet demuestra el uso de getServletInfo()";  
}
```

- Otro método interesante en Servlets es **sendRedirect()** que se utiliza para enviar al usuario a otra página o recurso. Va asociado a la respuesta del servidor. Su formato es:

```
// Redirige al cliente a otra página (puede ser interna o externa)  
response.sendRedirect("otraPagina.jsp");
```



3. JSP (Java Server Pages)

- JSP es una **tecnología basada en Java** que simplifica el proceso de desarrollo de **sitios web dinámicos**.
- Las Java Server Pages son **ficheros de texto** (normalmente con extensión **.jsp**) **que sustituyen a las páginas HTML tradicionales**. Los ficheros JSP contienen etiquetas HTML y código embebido que permite al diseñador de la página web acceder a datos desde código Java que **se ejecuta en el servidor**.



○ El objetivo de la tecnología JSP es dar soporte a la separación entre lógica de presentación y la de negocio o programación:

- ✓ Los diseñadores Web pueden diseñar y actualizar sus páginas sin necesidad de saber programar en Java.
- ✓ Los programadores pueden escribir código sin tener que preocuparse por el diseño de las páginas Web.



```
<HTML>
<HEAD>
<TITLE>Bienvenido al mundo JSP</TITLE>
</HEAD>
<BODY>
<H1>Bienvenido al mundo JSP</H1>
<%
java.util.Date fecha = new java.util.Date();
out.println("La fecha es : "+fecha);
%>
</BODY>
</HTML>
```




- JSP es una **tecnología híbrida** que al igual que ASP y PHP **puede incorporar scripts** para añadir código Java directamente a las páginas .jsp.
- La mayor parte de una página JSP es un patrón de texto HTML.
- Otra parte es donde se aloja el código Java, el cual va entre `<%
..... %>`.
- Esta página debe almacenarse con extensión .jsp
- Sobre todo, nos facilita escribir código sin tanto `out.println(" ");` para componer toda la página HTML.



3.1. Mecanismos de ejecución de JSP

- Las páginas JSP se deben alojar en directorios donde el Web Server las pueda localizar, normalmente en la misma jerarquía de directorio de las páginas HTML. Normalmente lo haremos en un paquete llamado “vistas” alojado dentro de “Web pages”.
- La primera vez que se realiza una petición de una página JSP, el contenedor Web convierte el fichero JSP en un servlet que pueda responder a la petición HTTP.



```
<HTML>
```

```
<HEAD><TITLE>Rapel</TITLE></HEAD>
```

```
<BODY>
```

Hoy tendrás un día :

```
<% if (Math.random() < 0.5) { %>
```

Buenísimo SUERTE increíble!

```
<% } else { %>
```

Malísimo GAFFE total!

```
<% } %>
```

```
</BODY>
```

```
</HTML>
```



3.2. Variables implícitas en JSP

- El contenedor JSP exporta un número de objetos internos para su uso desde las páginas .jsp.
- Estos objetos son asignados automáticamente a variables que pueden ser accedidas desde elementos de script de JSP.
- La siguiente tabla muestra los objetos disponibles y la API a la que pertenecen.



Objeto	Clase o Interfaz	Descripción
page	<i>javax.servlet.jsp.HttpJspPage</i>	Instancia del servlet de la página .jsp.
config	<i>javax.servlet.ServletConfig</i>	<i>Datos de configuración del servlet.</i>
request	<i>javax.servlet.http.HttpServletRequest</i>	<i>Petición. Parámetros incluidos.</i>
response	<i>javax.servlet.http.HttpServletResponse</i>	<i>Respuesta.</i>
out	<i>javax.servlet.jsp.JspWriter</i>	<i>Stream de salida para el contenido de la página.</i>
session	<i>javax.servlet.http.HttpSession</i>	<i>Datos específicos de la sesión de usuario.</i>
application	<i>javax.servlet.ServletContext</i>	<i>Datos compartidos por todas las páginas.</i>
pageContext	<i>javax.servlet.jsp.PageContext</i>	<i>Contexto para la ejecución de las páginas.</i>
exception	<i>java.lang.Throwable</i>	<i>Excepción o error no capturado.</i>



- **request:** El objeto `HttpServletRequest` asociado con la petición.
- **response:** El objeto `HttpServletResponse` asociado con la respuesta que se envía al navegador.
- **out:** El objeto `JspWriter` asociado al flujo de salida de la respuesta.
- **session:** El objeto `HttpSession` asociado con la sesión de cierto usuario. Esta variable sólo tiene sentido si la página JSP forma parte de una sesión HTTP



- **application:** El objeto ServletContext de la aplicación Web.
- **config:** El objeto ServletConfig asociado con el servlet para esta página JSP.
- **pageContext:** Este objeto encapsula el entorno de una petición para esta página JSP.
- **page:** Esta variable es equivalente a la variable this.
- **exception:** Objeto que fue lanzado por alguna otra página JSP. Solo disponible en una página de error.



Los métodos más utilizados para trabajar con atributos son:

Método	Descripción
<code>setAttribute(key, value)</code>	Asocia un atributo con su valor correspondiente.
<code>getAttributeNames()</code>	Retorna los nombres de todos los atributos asociados con la sesión.
<code>getAttribute(key)</code>	Retorna el valor del atributo asociado con <i>key</i> .
<code>removeAttribute(key)</code>	Borra el atributo asociado con <i>key</i> .



3.3. Elementos de una JSP

- Los elementos están empotrados entre las **marcas** `<% %>`.
- Cualquier otro texto en la página JSP se considerará parte de la respuesta y se copiará textualmente al flujo de respuesta HTTP.
- Hay 5 tipos de elementos de Script.
 - ✓ **Comentarios:** `<%-- comentario --%>`
 - ✓ **Etiqueta de directiva:** `<%@ directiva %>`
 - ✓ **Etiqueta de declaración:** `<%! declaración %>`
 - ✓ **Etiqueta de scriptlet:** `<% código %>`
 - ✓ **Etiqueta de expresión:** `<%= expresión %>`.



Comentarios

- Hay 3 tipos de comentarios en una página JSP:
 - Comentarios HTML. `<!-- comentario. HTML -- >`
 - Comentarios de página JSP. `<%-- comentario. --%>`
 - Comentarios Java. `<% /* comentario */ %>`.



Scriptlet

- Se utilizan para encapsular código Java que será insertado en el servlet correspondiente a la página .jsp. `<% scriptlet %>`
- Los scriptlets pueden contener cualquier tipo de sentencias Java, pudiendo dejar abiertos uno o más bloques de sentencias que deberán ser cerradas en etiquetas de scriptlets posteriores.
- Se pueden incluir sentencias condicionales if-else if-else,



```
<% if (numero < 0) { %>
```

```
    <p>No se puede calcular el factorial de un número negativo.</p>
```

```
<% } else if (numero > 20) { %>
```

```
    <p>Argumentos mayores que 20 causan un desbordamiento de pila.</p>
```

```
<% } else { %>
```

```
    <p align=center><%= x %>! = <%= factorial (x) %></p>
```

- Se pueden incluir sentencias de iteración: for, while y do/while

```
<table>
```

```
<tr> <th> <i>x</i> </th> <th> <i>x</i>! </th> </tr>
```

```
<% for (long x = 0; x <=20; ++x) { %>
```

```
<tr> <td> <%= x %> </td> <td> <%= factorial (x) %> </td> </tr>
```

```
<% } %>
```

```
</table>
```




Declaraciones

- Las declaraciones se usan para definir variables y métodos específicos para una página .jsp.
- Las variables y los métodos declarados son accesibles por cualquier otro elemento de script de la página, aunque estén codificados antes que la propia declaración.
- La sintaxis es:

`<%! declaracion %>`



```
<%! private int x = 0, y = 0; private String unidad="cm"; %>
```

```
<%! static public int contador = 0; %>
```

```
<%! public long factorial (long numero)
{
if (numero == 0) return 1;
else return numero * factorial(numero - 1);
} %>
```

Otros ejemplos:

```
<%! public int Contador = 0; %>
<%! public void incrementa(int cantidad){
Contador=Contador+cantidad;}
%>.
```



- Un uso importante de la declaración de métodos es el manejo de eventos relacionados con la **inicialización y destrucción de la página .jsp**. La inicialización ocurre la primera vez que el contenedor JSP recibe una petición de una página .jsp concreta.
- La destrucción ocurre cuando el contenedor JSP descarga la clase de memoria o cuando el JSP se reinicia.
- Estos eventos se pueden controlar sobrescribiendo dos métodos que son llamados automáticamente: **jspInit()** y **jspDestroy()**.



```
<%! public void jspInit () {  
// El código de inicialización iría aquí.  
}%>
```

```
.....  
<%! public void jspDestroy()  
{  
// El código de finalización iría aquí.  
}  
%>
```

Obviamente su codificación es opcional.



Expresiones

- Contiene código Java que se evalúa durante la petición HTTP. El resultado de la expresión se incluye en el flujo de respuesta HTTP.
- Las expresiones están ligadas a la generación de contenidos. Se pueden utilizar para imprimir valores de variables o resultados de la ejecución de algún método. Todo resultado de una expresión es convertido a un String antes de ser añadido a la salida de la página.
- **Las expresiones no terminan con un punto y coma (;).**

`<%= expresión %>`



<%= (horas < 12)? "AM" : "PM" %>

*Diez es <%= (2*5) %>*

Bienvenido Sr. <%=name %>

Fecha actual: <%= new java.util.Date() %>



Directivas

- Especifica una información que **afectará** a la **traducción** de la página JSP.
- Conjunto de etiquetas que contienen instrucciones para el contenedor JSP con información acerca de la página en la que se encuentran. **No producen ninguna salida visible**, pero afectan a las propiedades globales de la página .jsp que tienen influencia en la generación del servlet correspondiente.
- Las directivas JSP van encerradas entre `<%@ .... %>`.



```
<%@ page session="false" %>  
<%@ page import = "java.util.Date" %>  
<%@ include file = "copyright.html " %>
```



Directiva page

La directiva page nos permite definir uno o más de los siguientes atributos:

- ✓ **import**="package.class" o
import="package.class1,...,package.classN".

Esto nos permite especificar los **paquetes que deberían ser importados**. Por ejemplo:

```
<%@ page import="java.util.*" %>
```

El atributo import es el único que puede aparecer múltiples veces.



Directiva page

- ✓ **contentType="MIME-Type"** o **contentType="MIME-Type; charset=Character-Set"**

Esto **especifica el tipo MIME de la salida**. El valor por defecto es text/html. Por ejemplo, la directiva:

```
<%@ page contentType="text/plain" %>
```

tiene el mismo valor que el scriptlet

```
<% response.setContentType("text/plain");%>
```



Directiva page

✓ **session="true|false".**

Un valor de true (por defecto) indica que la variable predefinida session (del tipo HttpSession) debería unirse a la sesión existente si existe una, si no existe se debería crear una nueva sesión para unirla. Un valor de false indica que no se usarán sesiones, y los intentos de acceder a la variable session resultarán en errores en el momento en que la página JSP sea traducida a un servlet.



Directiva page

✓ **buffer**="sizekb|none".

Esto especifica el tamaño del buffer para el JspWriter out. El valor por defecto es específico del servidor, debería ser de al menos 8kb.

✓ **autoflush**="true|false".

Un valor de true (por defecto) indica que el buffer debería descargarse cuando esté lleno. Un valor de false, raramente utilizado, indica que se debe lanzar una excepción cuando el buffer se sobrecargue.



Directiva page

✓ **extends**="package.class".

Esto indica la superclase del servlet que se va a generar. Debemos usarla con extrema precaución, ya que el servidor podría utilizar una superclase personalizada.

✓ **info**="message".

Define un string que puede usarse para ser recuperado mediante el método `getServletInfo`.



Directiva page

✓ **errorPage="url".**

Especifica una página JSP que se debería procesar si se lanzara cualquier Throwable pero no fuera capturado en la página actual.

✓ **isErrorPage="true|false".**

Indica si la página actual actúa o no como página de error de otra página JSP. El valor por defecto es false.



Como resumen, los posibles valores de los atributos se muestran a continuación.

Atributo	Valor	Valor por defecto
info	Cadena de texto.	<i>""</i>
language	Nombre del lenguaje de script.	<i>"java"</i>
contentType	Tipo MIME y conjunto de caracteres.	<i>"text/html"</i> <i>"ISO-8859-1"</i>
extends	Nombre de clase.	<i>Ninguno</i>
import	Nombres de clases y/o paquetes.	<i>java.lang,</i> <i>java.servlet,</i> <i>javax.servlet.http,</i> <i>javax.servlet.jsp</i>
session	Booleano.	<i>"true"</i>
buffer	Tamaño del buffer o "none".	<i>"8kb"</i>
autoFlush	Booleano.	<i>"true"</i>
isThreadSafe	Booleano.	<i>"true"</i>
errorPage	URL local.	<i>Ninguno</i>
isErrorPage	Booleano.	<i>"false"</i>



```
<HTML>
<HEAD>
<TITLE>Mi primera página JSP</TITLE>
</HEAD>
<BODY BGCOLOR="#FFFFFF">
<%@ page info="Probando página JSP....." %>
<%@ page language="java" %>
<%@ page contentType="text/plain; charset=ISO-8859-1" %>
<%@ page import="java.util *, java.awt. *" %>
<%@ page session="false" %>
<%@ page buffer="12kb" %>
<%@ page autoFlush="true" %>
<%@ page isThreadSafe="false" %>
<%@ page errorPage="/webapps/varios/error.jsp" %>
<%@ page isErrorPage="false" %>
Hola a todos!
</BODY>
</HTML>
```



Directiva include

- Esta directiva nos **permite incluir ficheros en el momento en que la página JSP es traducida a un servlet.**
- Los ficheros pueden ser documentos estáticos como páginas HTML, o documentos dinámicos como páginas JSP.
- Facilita la reutilización de un documento en diversas páginas.
- El fichero incluido es identificado mediante una URL y la directiva tiene el efecto de remplazarse a sí misma con el contenido del archivo especificado. Puede ser utilizada para incluir cabeceras y pies de páginas comunes a un desarrollo dado.
Ejemplo: `<%@ include file="url relativa" %>`.



3.4. Gestión de excepciones en JSP

○ Cuando surge un error durante la ejecución de una página .jsp, podemos reaccionar de tres modos distintos :

1. El **comportamiento por defecto** es el de **mostrar un mensaje en la ventana del navegador**.
2. También se puede **utilizar el atributo `errorPage`** de la directiva `page` para especificar una página de error alternativa

```
<%@ page errorPage="errorpage.jsp" %>
```

Y hace falta otra página, indicando que es una página de error.



```
<%@ page isErrorPage="true" %>
```

```
<h1> El error es <%= exception.getMessage() %>
```

```
<a href="anteriorPagina.jsp" >Volver</a>.
```

En las siguientes diapositivas se muestra un ejemplo de este tipo.



```
<!-- JSPVotacion.jsp -->
<html>
<head>
<title>
JSP Votacion
</title>
</head>
<body>
<%@ page errorPage="paginaerror.jsp" %>
<%! String valor;%>
<% valor=request.getParameter("partido");%>
<% if(valor!=null){%>
    <p>GRACIAS POR VOTAR</p>
    HA VOTADO : <%=valor%>
<%} else throw new Exception("Debes elegir un partido");%>
</body>
</html>
```



```
<!-- paginaerror.jsp -->
<html>
<head>
<title>
Pagina de Error
</title>
</head>
<body bgcolor="red">
<%@ page isErrorPage="true" %>
<h1><%= exception.getMessage() %> </h1>
<a href="ofertaPartidos.htm" >Volver</a>.
</body>
</html>
```



3. La tercera opción es **utilizar el mecanismo de manejo de excepciones de Java** usando scriptlets.

En la siguiente diapositiva se muestra un ejemplo.



```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>JSP Page</title>
  </head>
  <BODY BGCOLOR="#FFFFFF">
    <%! int x = 14; %>
    <%! public long factorial (long numero) throws IllegalArgumentException {
      if ((numero < 0) || (numero > 20))
        throw new IllegalArgumentException("Fuera de Rango");
      if (numero == 0) return 1;
      else return numero * factorial(numero - 1); } %>
    <% try {
      long resultado = factorial (x); %>
      <p align=center> <%= x %>! = <%= resultado %></p>
    <% } catch (IllegalArgumentException e) { %>
      <p>El argumento para calcular el factorial esta fuera de rango.</p>
    <% } %>
  </body>
</html>
```



3.5. Acciones en JSP

- Las acciones JSP usan construcciones de sintaxis XML para controlar el comportamiento del motor de servlets.
- Podemos **insertar un fichero dinámicamente, reutilizar componentes JavaBeans** (encapsular varios objetos en un único objeto, para hacer uso de un solo objeto en lugar de varios más simples) o **reenviar al usuario a otra página**.
- Como en XML, los nombres de elementos y atributos son sensibles a las mayúsculas.



○ Tipos de Acciones:

- ✓ **jsp:include** - Incluye un fichero en el momento de petición de esta página.
- ✓ **jsp:useBean** - Encuentra o instancia un JavaBean.
- ✓ **jsp:setProperty** - Selecciona la propiedad de un JavaBean.
- ✓ **jsp:getProperty** - Inserta la propiedad de un JavaBean en la salida.
- ✓ **jsp:forward** - Reenvía a una nueva página.



Acción jsp:include

- `<jsp:include page="localURL" flush="true" />`
- Esta acción nos **permite insertar ficheros en una página que está siendo generada.**
- Al contrario que la directiva include, que inserta el fichero en el momento de la conversión de la página JSP a un Servlet, esta acción inserta el fichero en el momento en que la página es solicitada.



Acción jsp:include

- Proporciona una forma sencilla de incluir contenido generado por otro documento local en la salida de la página actual. En contraposición con la etiqueta forward, include transfiere el control temporalmente.
- El valor del atributo flush, determina si se debe vaciar el buffer de la página actual antes de incluir el contenido generado por el programa incluido.



- Se pueden incluir parámetros para llevarlos al otro jsp

```
<jsp:include page="localURL" flush="true">  
  <jsp:param name="nombreParametro1"  
    value="valorParametro1" />  
  ...  
  <jsp:param name="nombreParametroN"  
    value="valorParametroN" />  
</jsp:include>
```



Acción `jsp:forward`

- `<jsp:forward page="localURL" />`
- Se usa para **transferir el control de una página .jsp a otra localización**. Cualquier código generado hasta el momento se descarta, y el procesamiento de la petición se inicia en la nueva página.
- El valor del parámetro `page`, puede ser un documento estático, un servlet u otra página JSP.



3.6. Comunicación entre páginas JSP

- Existen tres formas fundamentales de **comunicar datos** entre páginas JSP o servlet.

1. **Mediante enlaces con parámetros.**

```
<a href="/detalle.jsp?codigo=<%=codigo%>">
```

posteriormente, en detalle.jsp se extrae el parámetro

```
<%=parametro=request.getParameter("codigo");%>
```

2. **Mediante formularios**, de igual manera, investigando los parámetros transmitidos.



3. Implementando sesiones.

- Como ya sabemos, las sesiones son espacios de memoria comunes a todos los servlets o jsp que forman el sitio web. En JSP, existe predefinido un único objeto sesión para todo el sitio web, pero con la posibilidad de crear tantos atributos como sea necesario.
- Se verán más adelante.